Elección del gestor de mensajería

Estudiante: Samuel Cobo

Objetivo. Comparar cinco gestores de mensajería para elegir el más adecuado para los pedidos, pagos y confirmaciones de BikeStore.

Comparación tecnológica:

Gestor	Ventajas	Limitaciones para BikeStore
RabbitMQ	Enruta mensajes a varias colas; usa confirmaciones y Dead-Letter Exchanges. Tiene soporte directo en Spring Boot .	Hay que configurar exchanges, colas, vínculos y reintentos. Es trabajo adicional, pero manejable.
Apache Kafka	Permite manejar muchos eventos y conservarlos para leerlos nuevamente. Útil para análisis e historial de compras.	Requiere trabajar con tópicos, particiones y offsets. Para esta práctica añade complejidad.
Apache ActiveMQ	Se integra bien con Java/JMS. Ofrece reentrega de mensajes y cola de fallidos (DLQ).	También funciona, pero su configuración JMS no simplifica especialmente nuestro ejemplo frente a RabbitMQ.
Amazon SQS	Servicio de colas administrado por AWS: no necesitamos mantener un bróker propio; admite DLQ.	Depende de AWS, credenciales y conexión a la nube. Complica una demostración local en clase.
IBM MQ	Mensajería empresarial confiable, con integración entre plataformas y manejo de mensajes no entregados.	Su administración e infraestructura tienen más alcance del que necesita un prototipo universitario.

Elección: RabbitMQ:

Elegimos RabbitMQ porque el ejercicio necesita distribuir trabajos concretos y manejar fallos, no analizar millones de eventos. Se puede instalar localmente con Docker e integrar con Spring Boot, sin depender de una cuenta en la nube. Además, sus colas y Dead-Letter Exchanges permiten mostrar claramente qué sucede con cada pedido.

Implementación:

OrderProducer publicará el pedido en JSON; **PaymentWorker** intentará cobrar con 50 % de probabilidad de fallo y hasta tres reintentos. Si no se logra cobrar, se enviará el mensaje a la DLQ. Si el pago queda en **PAID**, se publicará un evento que **EmailWorker** consumirá para simular el correo de confirmación. Cada paso registrará pedidoId, timestamp y thread.

Fuentes:

- https://www.rabbitmq.com/docs/dlx
- https://kafka.apache.org/documentation/
- https://activemq.apache.org/components/classic/documentation/message-redelivery-and-dlq-handling
- https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html
- https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=mq
- https://docs.spring.io/spring-boot/reference/messaging/amqp.html